

CSEN 296B-2: Week 1.2 Submission 1 — Architecture Decision Log

Student Name: Alex Shirazi

Date: April 1, 2026

Project: AstraNotes

Chosen Technical Path: Python

1. Architecture Decision Log

Initial Prompt (Weaker)

Act as a software architect. Design a backend for AstraNotes that lets users save and organize their notes.

What the first response did well

- It identified that AstraNotes needs a data model for notes and some form of persistent storage.
- It suggested using a simple file-based or database-backed approach for saving notes.

What the first response did poorly

- It did not define clear module boundaries or component responsibilities.
 - It treated storage and business logic as one concern with no separation.
 - It had no discussion of error handling, input validation, or edge cases.
 - It ignored privacy concerns entirely — no mention of how private notes should be handled.
 - It did not address testability or how the design could evolve over the quarter.
-

Stronger Prompt

Act as a senior software architect specializing in modular Python systems. Design a layered storage architecture for AstraNotes, a personal note-taking application. Think step by step. Define: (1) the note data model including metadata fields, (2) a repository interface that abstracts persistence operations, (3) a file-based storage adapter for local disk persistence, (4) an error-handling strategy for invalid save, load, and delete operations, (5) a privacy boundary that protects notes marked as private, and (6) how this architecture remains testable and extensible so that new note types such as voice or image notes can be added later without rewriting the storage layer. Avoid unnecessary external dependencies and keep the design appropriate for a solo-developer quarter-long university project.

Excerpt from the Stronger AI Response

Layered Architecture for AstraNotes

Note Model: Define a `Note` dataclass with fields: `id` (UUID), `title` (str), `body` (str), `is_private` (bool), `created_at` (datetime), `modified_at` (datetime), and `tags` (list of str). This gives each note a stable identity and rich metadata from the start.

Repository Interface: Create an abstract `NoteRepository` class with methods: `save(note)`, `get(note_id)`, `list_all()`, `update(note)`, and `delete(note_id)`. This decouples business logic from any specific storage mechanism.

File Storage Adapter: Implement `JsonFileRepository(NoteRepository)` that serializes notes to individual JSON files in a local directory. Each note maps to `{note_id}.json`. This avoids external database dependencies and keeps the project portable.

Error Handling: Wrap all I/O operations in try/except blocks. Define custom exceptions — `NoteNotFoundError`, `StoragePersistenceError`, `ValidationError` — so callers receive clear, actionable errors instead of raw tracebacks.

Privacy Boundary: Introduce a `PrivacyService` that encrypts the body of notes flagged as private before they reach the storage adapter, and decrypts on retrieval. Use the `cryptography` library's Fernet symmetric encryption with a user-provided key. The privacy logic lives outside the repository so it can be swapped or upgraded independently.

Extensibility: Because the repository interface accepts any object conforming to the `Note` base type, adding `VoiceNote` or `ImageNote` subclasses later requires only extending the model and adding a corresponding serializer — the storage adapter and privacy service remain unchanged.

Third Prompt Refinement

Refine the design so that AstraNotes supports text notes first but can later support voice or image notes without rewriting the storage layer. Show the major components, their responsibilities, and where validation, privacy, and version history should live. Include a component diagram description.

Final Architecture Decision

Component	Responsibility
Note Entity	Represents a single note with metadata (id, title, body, timestamps, tags, privacy flag)
Repository Interface	Defines abstract CRUD operations for note persistence, independent of storage backend
JSON File Storage Adapter	Implements the repository interface using local JSON files on disk
Privacy / Encryption Service	Encrypts and decrypts private note content before it reaches the storage layer
Validation Layer	Checks for invalid or incomplete state (empty title, malformed data) before any save operation
Version History Service	Tracks prior note states for recovery and audit by storing snapshots on update
Application Service (Note Manager)	Coordinates the workflow across validation, privacy, storage, and versioning components

Why I Would Defend This Architecture

I would defend this architecture because it cleanly separates each concern into its own module, which makes it possible to test, review, and evolve each piece independently across the quarter. The repository interface means I can swap out JSON file storage for SQLite or a cloud backend later without touching business logic. Keeping privacy logic in its own service ensures that encryption is not tangled into the UI or storage code, which reduces the risk of accidental data exposure.

2. Architecture Verdict

The final architecture is stronger than the first attempt because it makes boundaries between components explicit, eliminates hidden coupling between storage and business logic, and gives me a defensible path from backlog items to implementation. The weaker prompt produced a flat, script-style answer that would have been difficult to extend, test, or explain in a design review. The stronger prompt forced the AI — and forced me — to reason about modularity, failure modes, and long-term maintainability before writing any code.

3. Short Reflection

The biggest takeaway from this lab was that improving a prompt is really the same as improving your engineering thinking. The weaker prompt gave me a surface-level answer I could not defend in a review. By adding constraints around modularity, error handling, privacy, and extensibility, the stronger prompt forced both me and the AI to produce something I could actually trace forward into backlog items, design documents, and testable code.